>>> import python

Aula 04

Python

Comentários · conversão de tipo · formatação de strings

2026.1

aula_04.py

```python
# comentário
nota = 9.666
idade = int("20")

print(f"nota {nota:.2f}")

# comentários · conversão
# · formatação
```

>>> agenda

O que veremos hoje

01

Comentários

Documentar o código para humanos

02

Conversão de tipo

int, float, round e a lib math

03

Formatação

Interpolação, f-strings e format

01

>>> comentários

Comentários

Notas no código que o Python ignora — mas que ajudam quem lê.

O comentário tradicional

```
# isso é um comentário
nome = input(...)
```

tudo depois do # na linha é ignorado pelo Python.

Boa prática: comentário bom explica o porquê, não o óbvio. Evite “soma a e b” em cima de a + b.

Comentando várias linhas

Três aspas — simples ''' ou duplas """ — abrem e fecham um bloco de texto que pode ocupar várias linhas.

```
"""
   Comentário
   em
   bloco
"""

a = 5
```

Quando usar

- Explicações mais longas
- Documentar o topo de um arquivo
- Descrever funções (docstrings)

02

>>> conversão de tipo

Conversão de tipo

Transformando um valor de um tipo em outro: int, float e afins.

Convertendo para inteiro — int()

```
print(3.14)
print(int(3.14))
print(int(3.8))
print(int(-3))
print(int("3"))
print(int("g"))
```

Saída

```
3.14
3
3
-3
3
ValueError!
```

Atenção: int() corta a parte decimal (não arredonda) e falha com texto não numérico como "g".

Convertendo para decimal — float()

```
print(3)
print(float(3))
print(float("3.14"))
```

Saída

```
3
3.0
3.14
```

Repare: float() aceita inteiros e textos numéricos, adicionando a parte decimal (.0 quando não houver).

Arredondando — round()

```
print(round(3.14))
print(round(3.8))
print(round(3.139999, 2))
```

Saída

```
3
4
3.14
```

Diferença: diferente de int(), round() arredonda ao vizinho mais próximo. O segundo argumento define as casas decimais.

A biblioteca math

```
import math

print(math.ceil(3.14))
print(math.floor(3.8))
```

ceil → arredonda para cima (4)

floor → arredonda para baixo (3)

Guarde: para trabalhar com cálculos, a math é fundamental — traz raiz, potência, pi, seno e muito mais.

03

>>> formatação de strings

Formatação de strings

Montando textos com valores de variáveis dentro.

Concatenação e o print

```
print("a" + "b" + "c")
print("a", "b", "c")
```

Saída

```
abc
a b c
```

Diferença: o + gruda os textos; a vírgula passa vários argumentos ao print, que os separa com um espaço.

Inserindo valores no texto

Interpolar é colocar o valor de uma variável dentro de um texto. Há várias formas — vamos das mais antigas às atuais.

```
# com vírgulas (o print separa por espaço)
print("Texto número", num_um, "de", num_dois)

# com .format
print("Texto número {} de {}".format(num_um, num_dois))
```

O método %

```
texto = "Olá Mundo"
print("Bem-vindo! %s" % texto)
```

Saída

```
Bem-vindo! Olá Mundo
```

Importante: o operador % ainda funciona, mas não é a forma atual. Prefira f-strings no código novo.

O método .format

```
# por posição
print("{}, {}".format("Um", "Dois"))

# por nome
print("{nome}, {sobrenome}".format(
        nome="João", sobrenome="Victor"))
```

Saída

```
Um, Dois

João, Victor
```

F-strings

Python 3.6+ a maneira moderna e mais legível: um f antes das aspas e a variável entre chaves.

```
print(f'Texto {num_um} de {num_dois}')
```

f'...' ativa a interpolação;
{var} insere o valor.

Use esta: f-strings são o padrão recomendado hoje: curtas, rápidas e fáceis de ler.

Formatando números

Dentro das chaves, um : abre opções de formatação. O .2f mostra o número com duas casas decimais.

```
nota = 9.66666
print(f'{nota:.2f}')
```

Saída

```
9.67
```

Muito útil: ótimo para dinheiro e notas: {preco:.2f} garante sempre duas casas. Repare que arredonda o último dígito.

Executando código nas chaves

Dentro das chaves cabe qualquer expressão Python — inclusive conversões e contas.

```
nome = input("Seu nome: ")
idade = input("Sua idade: ")

print(f"{nome} terá {int(idade) + 1} anos no próximo ano.")
```

Repare: int(idade) + 1 é calculado na hora. Como input devolve texto, a conversão é indispensável.

Qual forma usar?

`% (porcentagem)`	**Legado**	Evite em código novo
`Concatenação com +`	**Ok para casos simples**	Verboso com muitas variáveis
`.format()`	**Ainda comum**	Útil com nomes e reuso
`f-string`	**Recomendado**	Curto, rápido e legível

Hora de praticar

1. Peça o nome do usuário e imprima uma mensagem de boas-vindas com f-string.

2. Solicite dois números, calcule a soma e mostre o resultado formatado numa frase.

3. Receba o nome de um produto, o preço e a quantidade, e imprima um recibo formatado.

Dica: no recibo, use {preco:.2f} para o valor e mostre o total (preço × quantidade).

O essencial da aula

`# / """ """`	Comentários de linha e de bloco
`int() float()`	Convertem entre número e texto
`round()`	Arredonda ao vizinho mais próximo
`math`	ceil, floor e outros cálculos
`f-string`	A forma atual de formatar textos
`:.2f`	Controla as casas decimais

```
>>> exit()
```

DÚVIDAS?

Pratique: **os três exercícios com f-strings**